

Disease Prediction System Using Machine Learning

**Submitted By:
Adithya Menon**

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Project Description	3
Updated Dataset	4
Environment Setup	5
Data Exploration	6
Data Preprocessing	7
Proposed System	8
Model Development	9-10
Disease Prediction Function	11
Conclusion	12

1. Project Description

MedAssist AI is an intelligent disease prediction system that predicts the most probable disease based on symptoms entered by the user. Unlike the previous milestone, the project has been upgraded to use a significantly larger disease-symptom dataset, enabling the prediction of a wider range of diseases.

During this milestone, the focus was on:

- Migrating to a larger disease dataset
- Data preprocessing and filtering
- Training ensemble machine learning models
- Implementing boosting algorithms
- Comparing model performance
- Feature importance analysis
- Preparing trained models for backend integration

2. Updated Dataset

Instead of the previous 41-disease dataset, the project now uses a much larger symptom-disease dataset.

Dataset Characteristics

Property	Value
Total Records	189,647
Original Diseases	773
Selected Diseases	257 (after filtering)
Total Features	377 Symptoms
Target Column	Disease

Dataset Processing

The original dataset contained many diseases with only one or two samples, which can negatively affect model performance.

Therefore,

- diseases with fewer than 200 samples were removed,
- highly imbalanced classes were filtered,
- only sufficiently represented diseases were retained for training.

This produced a cleaner and more balanced dataset for machine learning.

3. Environment Setup

The project was developed using

Software

- Python 3.11
- Visual Studio Code
- Google Colab
- Jupyter Notebook

Libraries

- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib
- Matplotlib

Hardware

Local System

- Intel Core i5 Processor
- 8 GB RAM

Since XGBoost required high computational resources, training was performed using **Google Colab**.

4. Data Exploration

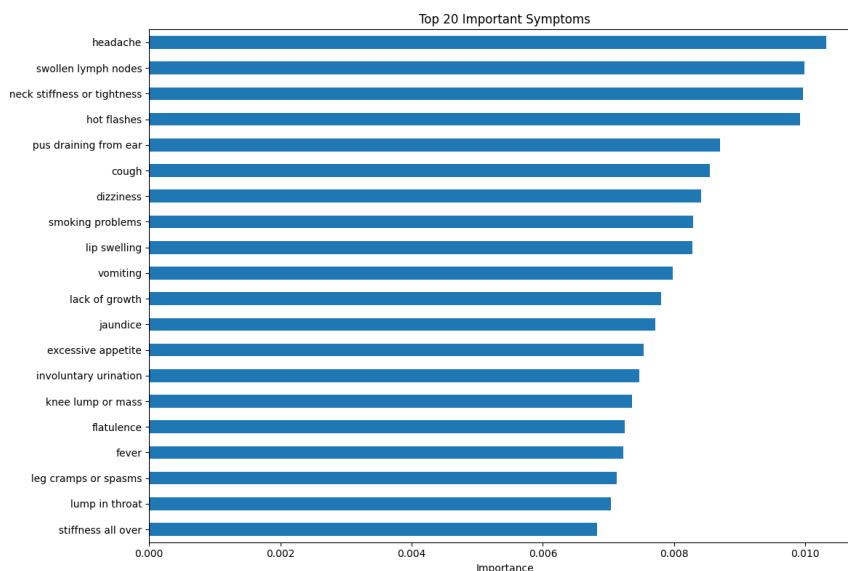
The updated dataset was analyzed before training.

The following analyses were performed:

- Dataset Shape
- Number of Diseases
- Number of Symptoms
- Missing Value Analysis
- Duplicate Record Detection
- Disease Frequency Distribution
- Feature Importance

Observations

- No missing values
- Binary symptom representation
- Large number of diseases
- Class imbalance present
- Filtering improved class distribution



5. Data Preprocessing

The following preprocessing steps were applied:

Label Encoding

Disease labels were converted into numerical values using LabelEncoder.

Feature Selection

Input:

377 binary symptom features

Target:

Disease

Train-Test Split

- 80% Training
- 20% Testing

using stratified sampling.

Model Serialization

Saved files include:

- Random Forest Model
- XGBoost Model
- Extra Trees Model
- Label Encoder
- Feature Columns
- Ensemble Weights

6. Proposed System

User Symptoms



Feature Vector Generation



Ensemble Machine Learning Models

(Random Forest + XGBoost + Extra Trees)



Soft Voting



Predicted Disease



Future Modules

Description

Medicine

Diet

Workout

Precautions

7. Model Development

The following machine learning models were implemented:

Base Models

- Random Forest
- Extra Trees
- XGBoost

Ensemble

Soft Voting Ensemble

where weighted probabilities from each classifier are combined to produce the final prediction.

```
=====
MODEL PERFORMANCE
=====
Random Forest Accuracy : 0.8194
XGBoost Accuracy       : 0.8194
ExtraTrees Accuracy    : 0.7190
=====

Ensemble Weights
{'rf': 0.3609852256515614, 'xgb': 0.36103815991729443, 'et': 0.27797661443114424}

=====
SOFT VOTING ENSEMBLE
=====
Accuracy : 0.8206
Precision : 0.8270
Recall    : 0.8206
F1 Score  : 0.8222
```

Boosting Models (Implemented)

- CatBoost
- LightGBM
- XGBoost

Hyperparameter tuning and comparison are currently under evaluation.

8. Model Evaluation

Models were evaluated using

- Accuracy
- Precision

- Recall
- F1 Score
- Classification Report

Individual Model Performance

Model	Accuracy
Random Forest	81.94%
XGBoost	81.94%
Extra Trees	71.90%

Soft Voting Ensemble

Metric	Score
Accuracy	82.06%
Precision	82.70%
Recall	82.06%
F1 Score	82.22%

Boosting Model Results (Current Progress)

The boosting notebook has been implemented, and model comparison is currently in progress.

Model	Accuracy
CatBoost	90.16%
Random Forest	89.68%
XGBoost	88.18%
LightGBM	87.72%
Extra Trees	87.58%

Note: These boosting results are preliminary and will be finalized after complete hyperparameter tuning and validation.

9. Disease Prediction Function

Overview

A prediction function was developed to simulate the final disease prediction workflow. The function accepts a list of symptoms from the user, converts them into a binary feature vector compatible with the trained models, and generates a disease prediction using the weighted soft voting ensemble.

Prediction Workflow

User Symptoms



Binary Feature Vector



Random Forest



XGBoost



Extra Trees



Weighted Soft Voting



Predicted Disease

Sample Prediction

Input Symptoms	Predicted Disease
Fever, Cough, Headache	Strep Throat

This demonstrates that the trained models can be integrated into the application to provide disease predictions based on user-entered symptoms. This section is a strong addition because it shows the transition from model training to an application-ready prediction pipeline.

10. Conclusion

During Milestone 2, the MedAssist AI disease prediction system was significantly enhanced by migrating to a larger augmented symptom–disease dataset and implementing multiple ensemble learning techniques. Tree-based machine learning models, including Random Forest, Extra Trees, and XGBoost, were trained and combined using a weighted Soft Voting Ensemble, achieving an overall accuracy of approximately **82%** on the test dataset.

In addition to model training and evaluation, a **symptom-based prediction function** was developed. This function converts user-entered symptoms into a binary feature vector, obtains prediction probabilities from each trained model, combines them using weighted soft voting, and returns the disease with the highest predicted probability. This prediction pipeline forms the core of the disease prediction system and is ready for backend integration.

Feature importance analysis was also performed to identify the most influential symptoms contributing to disease prediction, improving the interpretability of the model. The trained models, label encoder, feature columns, ensemble weights, and other preprocessing components were serialized using Joblib to support deployment.

Furthermore, a separate boosting pipeline was implemented to evaluate advanced boosting algorithms such as CatBoost, LightGBM, and XGBoost. Hyperparameter tuning and final model selection are currently in progress to identify the best-performing model for deployment.

The next phase of the project will focus on finalizing the optimized model, integrating it with the Django backend, developing symptom-based disease prediction APIs, and connecting the prediction engine with the frontend chatbot to provide an interactive healthcare assistance system.